Elevatex EFB

Electronic Flight Bag – SimBrief OFP viewer · v1.0.0

1. Overview

A frontend-only web app that displays a SimBrief Operational Flight Plan inside an iPad frame, styled after Apple's iPadOS design. No backend or database: the SimBrief API is public and supports CORS.

- React 19 + TypeScript
- Vite
- Lucide React (icons)
- Vanilla CSS

2. Getting started

```
npm install
npm run dev
npm run build
```

3. Project structure

```
src/
├── App.tsx                    main screen and state
├── App.css                    layout and components styles
├── index.css                  design tokens and reset
├── components/
│   ├── SettingsView.tsx       SimBrief ID input and verification
│   └── FlightPlanView.tsx     OFP display
├── hooks/
│   └── useDragScroll.ts       touch-like mouse drag scrolling
└── services/
    └── simbrief.ts            API calls, parsing and error mapping
```

4. SimBrief API

Numeric input is sent as `userid`, anything else as `username`.

```
const isNumeric = /^\d+$/.test(trimmed)
const queryParam = isNumeric
  ? `userid=${encodeURIComponent(trimmed)}`
  : `username=${encodeURIComponent(trimmed)}`
const url = `https://www.simbrief.com/api/xml.fetcher.php?${queryParam}&json=1`
```

Error handling

SimBrief answers errors with HTTP `400` but still sends a JSON body. The body is parsed first, then the status is mapped to a clear message.

```
const response = await fetch(url)
const data = await response.json().catch(() => null)

if (!data) {
  return { success: false, errorType: 'network_error', ... }
}

const status = data.fetch?.status || data.status || ''
```

SimBrief status	Meaning	errorType
Success	OFP available	–
No flight plan on file	Account exists, no OFP	no_ofp
Unknown UserID	Account not found	unknown_user
No JSON / fetch throws	Network issue	network_error

Parsed OFP model

```
export interface SimbriefOfp {
  flightNumber: string
  origin: FlightAirport
  destination: FlightAirport
  aircraft: FlightAircraft
  route: string
  altitude: string
  flightTime: string
  blockTime: string
  distance: string
  fuel: FlightFuel
  generatedDate: string
}
```

Formatting helpers

```
function formatAltitude(alt: unknown): string {
  const a = parseInt(String(alt), 10)
  if (isNaN(a)) return '--'
  if (a >= 10000) return `FL${Math.round(a / 100)}`
  return `${a.toLocaleString()} ft`
}

function formatSeconds(sec: unknown): string {
  const s = parseInt(String(sec), 10)
  if (isNaN(s)) return '--:--'
  const hrs = Math.floor(s / 3600)
  const mins = Math.floor((s % 3600) / 60)
  return `${hrs}h ${mins < 10 ? '0' : ''}${mins}m`
}
```

5. App state

The SimBrief ID is persisted in `localStorage` so the user does not have to type it again.

```
const STORAGE_KEY = 'elevatex_simbrief_id'

const [simbriefId, setSimbriefId] = useState<string>(() => {
  return localStorage.getItem(STORAGE_KEY) || ''
})
const [ofp, setOfp] = useState<SimbriefOfp | null>(null)
const [isLoadingOfp, setIsLoadingOfp] = useState<boolean>(false)
const [ofpError, setOfpError] = useState<string | null>(null)
```

The main screen renders one of four states:

- **Loading** – spinner while the API responds
- **Error** – message with Retry and Check Settings actions
- **OFP** – `FlightPlanView`
- **Empty** – prompt to configure or fetch

6. Settings

The user enters a Pilot ID or username, then the account is checked live against SimBrief. The ID is only saved if the account exists.

```
setIsVerifying(true)
const result = await verifySimbriefUser(trimmed)
setIsVerifying(false)
setVerification(result)

if (result.isValid) {
  onSaveSimbriefId(trimmed)
}
```

7. Displayed data

- Flight number, aircraft type and registration
- Departure / arrival ICAO, airport name, planned runway
- Flight time and block time
- Cruise altitude
- Block fuel and trip fuel (KGS / LBS)
- Filed route

8. Drag scrolling

`useDragScroll` lets the user scroll with a mouse drag like on a touch screen. Buttons and inputs are ignored so clicks still work.

```
const scrollRef = useDragScroll<HTMLElement>()

<section className="efb-body" ref={scrollRef}>
```

```
const handleMouseMove = (event: MouseEvent) => {
  if (!isMouseDown) return
  const deltaY = event.clientY - startY
  if (Math.abs(deltaY) > 4) {
    isDragging = true
  }
  container.scrollTop = startScrollTop - deltaY
}
```

9. Design

- Apple iPadOS dark mode colors (system backgrounds and labels)
- Inset grouped lists, like iOS Settings
- Translucent cards with `backdrop-filter` over the wallpaper
- Hidden scrollbars, wheel and drag scrolling

```
:root {
  --system-bg-base: #000000;
  --system-bg-secondary: #1c1c1e;
  --system-bg-tertiary: #2c2c2e;
  --label-primary: #ffffff;
  --label-secondary: #8e8e93;
  --system-blue: #0a84ff;
}
```

10. AI tools used

Type	Tool
IDE	Antigravity
Model	Gemini 3.8 Flash